

Maternas — Informe Técnico de Desarrollo



**Institución
Universitaria
de Envigado**

VIGILADA MINEDUCACIÓN

Chatbot RAG para Información y Orientación en Salud Materna

Informe técnico del sistema Convocatoria **890 de Minciencias** · Institución Universitaria de Envigado

Equipo de desarrollo

- **Juan Pablo Ríos Ortiz** — `jpriosso@correo.iue.edu.co`
- **Daniel Restrepo Villa** — `drestrepov@correo.iue.edu.co`
- **Cristian Troncoso Guerra** — `ctroncosog@correo.iue.edu.co`

Información del proyecto

Campo	Detalle
Proyecto	Agente inteligente basado en procesamiento de lenguaje natural para seguimiento materno en época pospandémica para un entorno de Telemedicina
Código del proyecto	COD_00-215
Institución	Institución Universitaria de Envigado
Versión evaluada	<code>5707d01</code> · rama <code>master</code>
Índice en producción	Configuración D
Vectores indexados	253.455
Fecha del informe	15 de agosto de 2026

1. Resumen ejecutivo

Maternas-Chatbot-Obstétrico es un chatbot conversacional basado en Recuperación Aumentada por Generación (RAG), orientado a madres gestantes y en período postparto, que responde consultas de salud materna en español sobre control prenatal, signos de alarma, medicamentos, nutrición, lactancia y salud mental perinatal. El sistema clasifica previamente la intención y el nivel de riesgo clínico de cada mensaje antes de generar una respuesta fundamentada en literatura médica indexada.

El proyecto busca aprovechar las **Tecnologías de la Información y las Comunicaciones (TIC)**, junto con estrategias de telesalud, como herramientas de apoyo para facilitar el acceso oportuno a información confiable en salud materna, especialmente en contextos donde existen barreras de acceso a los servicios de salud. A futuro, se proyecta que esta tecnología pueda contribuir al fortalecimiento de la educación, la orientación y la identificación temprana de situaciones de riesgo durante el embarazo y el posparto, en articulación con los servicios de salud y bajo criterios de seguridad clínica.

Esta iniciativa se alinea con el **Objetivo de Desarrollo Sostenible 3 (ODS 3)**, particularmente con la meta 3.1, orientada a reducir la mortalidad materna y avanzar hacia una atención sanitaria accesible y de calidad.

El sistema opera con costo marginal cercano a cero (APIs gratuitas de Groq/Cerebras, embedding local) sobre hardware de gama media, sin fine-tuning. A la fecha de este informe el desarrollo está en estado **funcional y probado en vivo**: expone tres interfaces (API FastAPI, UI Streamlit con panel de administración, bot de Telegram), un índice FAISS de 253.455 vectores en producción (Config D), una suite de 5 configuraciones de retrieval evaluadas experimentalmente con Ragas, y una batería de pruebas automatizadas (`pytest`). Las métricas de evaluación actuales (`faithfulness` 0.497, `answer_relevancy` 0.733 sobre baseline 0.558) muestran una recuperación de contexto todavía por debajo del sistema de referencia publicado, atribuible principalmente a la ausencia de los documentos fuente exactos del benchmark de evaluación en el índice de producción — una limitación conocida y documentada, no un hallazgo nuevo de este informe.

2. Contexto y objetivos

Problema que resuelve

En contextos de bajos recursos, las gestantes frecuentemente no tienen acceso rápido a orientación médica ante dudas cotidianas o síntomas de alarma. Maternas actúa como primer filtro informativo: clasifica la urgencia clínica de cada consulta y, cuando detecta riesgo medio o alto, escala mediante una notificación automática por correo electrónico, además de indicar siempre al usuario que busque atención profesional.

Alcance funcional

- Conversación en lenguaje natural sobre síntomas del embarazo, control prenatal, medicamentos, nutrición, lactancia, postparto y salud mental perinatal.
- Clasificación automática de intención (12 categorías) y de riesgo clínico (bajo/medio/alto, en cascada heurística → LLM).
- Recuperación de fragmentos de literatura médica (FAISS) y generación de respuestas con citas a la fuente.

- Escalamiento por correo electrónico ante riesgo medio/alto.
- Panel de administración para gestión del índice, configuración en caliente y monitoreo.

Usuarios objetivo

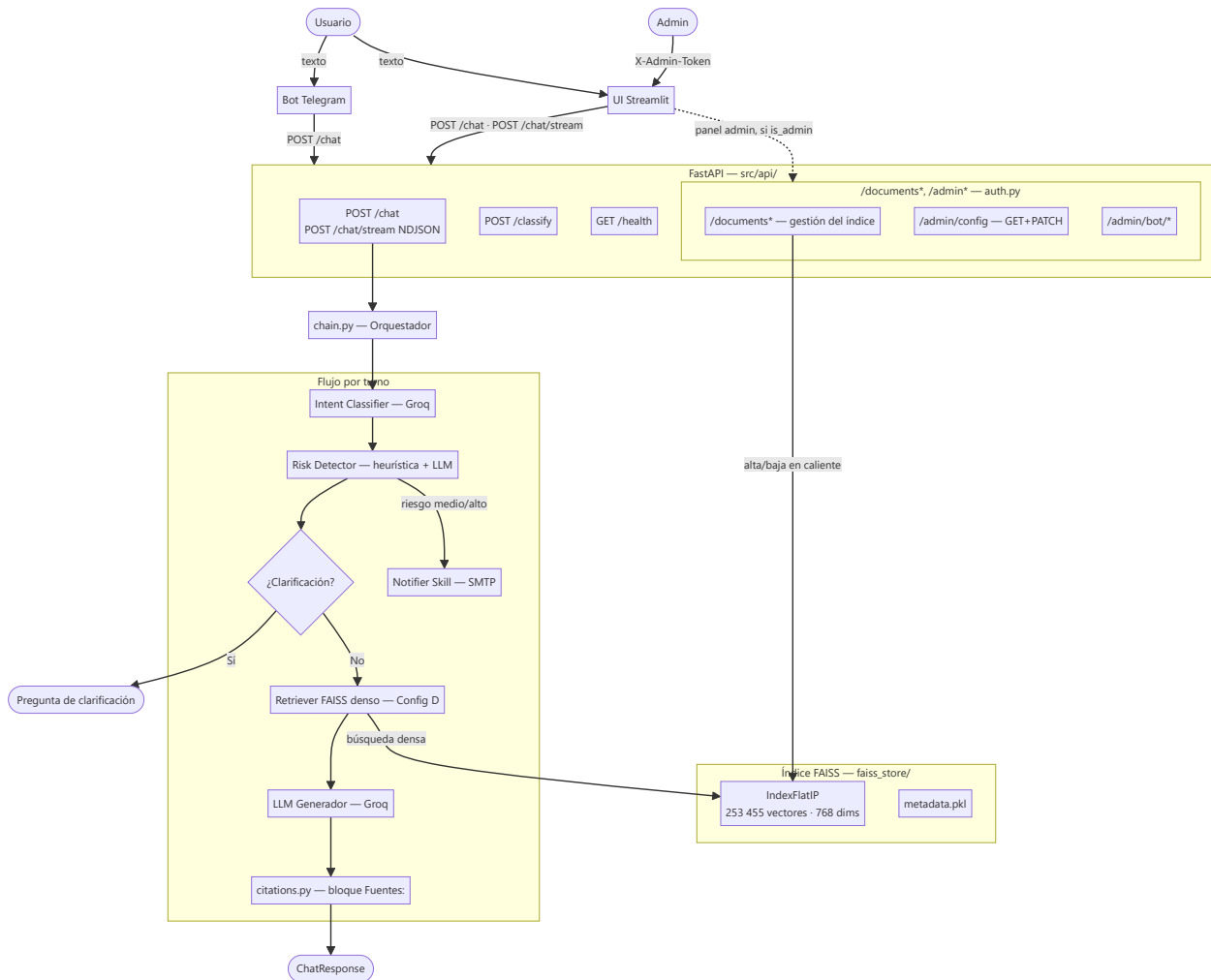
Gestantes y su red de apoyo (pareja, familiares) — el sistema está diseñado explícitamente para que terceros puedan consultar en representación o junto a la gestante (ver evidencia en la sección 6).

Restricciones del proyecto

Según `foragents/project_constraints.md`: sin QLoRA ni fine-tuning en esta fase, priorizando simplicidad, rapidez de implementación y costo cercano a cero, sobre hardware disponible (AMD Ryzen 5, RTX 2050, 16 GB RAM).

3. Arquitectura de la solución

Diagrama de alto nivel



Componentes principales

Componente	Responsabilidad
Bot Telegram	Cliente ligero: reenvía a <code>POST /chat</code> , historial en RAM, scheduler de check-ins
UI Streamlit	Chat público + panel admin (Dashboard/Documentos/Métricas/Configuración/Consola)
FastAPI	Valida requests, carga FAISS en <code>lifespan</code> , expone endpoints públicos y administrativos
chain.py	Orquestador del turno: intent → risk → clarificación → notificación → retrieval → generación → citas
Intent Classifier	12 categorías, zero-shot vía Groq, con fallback heurístico

Componente	Responsabilidad
Risk Detector	3 niveles, heurística instantánea + LLM de confirmación, consciente del historial de la conversación
Retriever (Config D)	Búsqueda densa FAISS pura sobre <code>medmcqa</code> + <code>medqa_*</code> + <code>maternaqaes_lm</code>
Notifier Skill	Alerta SMTP ante riesgo medio/alto

Arquitecturas evaluadas

El retriever pasó por cinco configuraciones sucesivas, cada una medida con el mismo protocolo de evaluación Ragas antes de decidir el cambio (detalle completo en `foragents/retrieval_arquitecturas_configs.md` y `foragents/qa_technical.md`):

Config	Descripción	Resultado de la decisión
A — FAISS puro	Búsqueda densa sobre todo el índice sin distinción de fuente	Descartada: casos clínicos de Multiclinsum contaminaban el contexto
B — FAISS + BM25	Capa densa filtrada por fuente + BM25 léxico solo para Multiclinsum	Mejóro <code>faithfulness</code> (+41%) y latencia (−9%) vs A
C — + corpus obstétrico ES	Se añadió <code>maternaqaes_lm</code> (corpus colombiano)	Salto grande en <code>context_recall</code> (0.067→0.452) y <code>faithfulness</code> (+27%)
D — sin <code>textbook</code> / <code>multiclinsum</code> (<i>producción actual</i>)	Se removieron 127.290 vectores (33,4% del índice) por riesgo de licencia no resuelto (18 textbooks EN sin licencia de reuso identificable, MultiClinSum sin garantía de desidentificación documentada)	Medido primero contra C antes de ejecutar: deltas dentro del ruido estadístico (± 0.25 std, N=14) — sin pérdida medible de calidad. <code>bm25_index.py</code> se eliminó por quedar sin corpus que indexar
E — Config D + HyDE	Genera un párrafo hipotético vía LLM antes de embedder la query	Descartada. Ninguna métrica mejoró de forma concluyente (deltas dentro del ruido); costo real: +1,27 s de latencia por turno y agotamiento de cuota diaria de Groq a mitad de la corrida de evaluación

La decisión de remover `textbook` / `multiclinsum` (Config D) y de no adoptar HyDE (Config E) ilustra el patrón de trabajo del equipo: cada cambio de arquitectura se midió con Ragas sobre el mismo conjunto de pares *antes* de aplicarse en producción, en vez de decidirse a priori.

4. Stack tecnológico

Capa	Tecnología	Versión
Lenguaje	Python	3.12.7
API backend	FastAPI + uvicorn	0.115.6 / 0.32.1
Interfaz web	Streamlit	1.41.1
Bot de mensajería	python-telegram-bot	21.10
LLM generador	<code>llama-3.3-70b-versatile</code> (Groq API)	—
LLM evaluador (judge)	<code>gemma-4-31b</code> (Cerebras API)	—
Embedding	<code>intfloat/multilingual-e5-base</code> (768 dims, ES/EN/ZH)	—
Vector store	FAISS <code>IndexFlatIP</code>	1.9.0
Orquestación LLM	LangChain (text splitters)	0.3.13
Evaluación	Ragas	0.2.12
Validación de configuración	Pydantic Settings	2.14.1
Cifrado de datos en reposo	<code>cryptography</code> (Fernet)	—

Datasets indexados: **MedMCQA** (187.005 preguntas médicas EN, Apache 2.0), **MedQA** (USMLE/Taiwan/Mainland, MIT) y **MaternaQA-es LM** (5.353 sub-chunks, corpus obstétrico colombiano, único dataset en español específico del dominio).

5. Setup

Pasos verificados para levantar el sistema en local (`README.md` y `docs/DOCUMENTACION.md` §14).

Instalación

```
# 1. Clonar el repositorio
git clone https://github.com/elrios893/maternas-rag.git
cd maternas-rag

# 2. Crear entorno virtual
python -m venv venv
.\venv\Scripts\activate      # Windows
# source venv/bin/activate   # Linux/macOS

# 3. Instalar PyTorch con soporte CUDA (si hay GPU NVIDIA – recomendado para ingesta)
pip install torch==2.5.1+cu121 --index-url https://download.pytorch.org/whl/cu121

# 4. Instalar el resto de dependencias
# (requirements.txt pinea sentence-transformers==2.7.0 – la versión 3.3.1
# producía un cuelgue silencioso al importar junto con torch)
pip install -r requirements.txt

# 5. Configurar variables de entorno
cp .env.example .env
# Editar .env con GROQ_API_KEY y demás valores requeridos
```

Arrancar el sistema

```
# Terminal 1 – API FastAPI
python -m uvicorn src.api.main:app --port 8080 --reload

# Terminal 2 – UI Streamlit
streamlit run src/ui/app.py

# Terminal 3 – Bot Telegram (opcional, requiere la API corriendo)
python src/bot/maternas_bot.py
```

- UI Streamlit: <http://localhost:8501>
- API docs (Swagger): <http://localhost:8080/docs>

Ingesta del índice FAISS (one-time, ~5h con GPU — no se ejecuta en este informe, ver sección 6.1)

```
python src/ingestion/run_ingestion.py
# o por dataset individual:
python -m src.ingestion.ingest_medmcqa
python -m src.ingestion.ingest_medqa
python -m src.ingestion.ingest_maternaqaes_lm
```

Tests

```
./venv/Scripts/python.exe -m pytest -q
```

Suite en `tests/` (clasificadores, gestión de documentos, config editable, supervisor del bot, citas, chat streaming, usuarios activos). Los fixtures de `tests/conftest.py` evitan cargar el índice FAISS real (~780 MB) y resetean los singletons de clientes Groq cacheados entre tests.

6. Proceso de desarrollo y evidencia de ejecución

Cada subprocesso sigue el formato *narrativa corta* → *evidencia visual* → *observación técnica*. La evaluación en vivo se realizó el 15 de agosto de 2026 contra la API (`localhost:8080`), la UI Streamlit (`localhost:8501`) y el bot de Telegram real (`@MaternasAssistant_bot`), con el índice FAISS de producción (253.455 vectores) cargado. Para cada caso de prueba se abrió una sesión nueva (pestaña nueva de Streamlit, o chat reiniciado de Telegram), de modo que el aviso de tratamiento de datos y el estado de riesgo partieran limpios en cada uno.

6.1 Ingestión

La ingestión puebla el índice FAISS a partir de los datasets crudos y **no se ejecuta en vivo** para este informe: reconstruir o alterar el índice de producción toma horas (~5h con GPU) y puede dejarlo en un estado inconsistente. Se documenta con el código real y los registros de ejecuciones anteriores.

`src/ingestion/run_ingestion.py` orquesta tres scripts idempotentes (Multiclinsum, MedMCQA, MedQA) que pueden relanzarse desde el último checkpoint si el proceso se interrumpe. Cada dataset pasa por `formatters.py` (7 formateadores, uno por fuente) y luego por `chunkers.py`, que decide la estrategia de chunking según `source_dataset`:


```

src/ingestion/run_ingestion.py

1  """run_ingestion.py - Orquestador de ingestión completa.
2
3  Ejecuta en orden:
4  1. Multiclinsum (summaries + fulltexts)
5  2. MedMCQA (train + dev)
6  3. MedQA (US + Taiwan + Mainland + textbooks EN)
7
8  Cada script es idempotente: si una fase ya fue completada la salta.
9  Si el proceso se interrumpe, al relanzar continúa desde el último checkpoint.
10 """
11
12 def main(only: str | None = None) -> None:
13     run_all = only is None
14     run_multiclin = run_all or only == "multiclinsum"
15     run_medmcqa = run_all or only == "medmcqa"
16     run_medqa = run_all or only == "medqa"
17
18     if run_multiclin:
19         from src.ingestion.ingest_multiclinsum import main as run_multiclinsum
20         run_multiclinsum()
21
22     if run_medmcqa:
23         from src.ingestion.ingest_medmcqa import main as run_mmc
24         run_mmc()
25
26     if run_medqa:
27         from src.ingestion.ingest_medqa import main as run_mq
28         run_mq()
29
30     print("-" * 60)
31     print("INGESTIÓN COMPLETA")
32     print("-" * 60)

```

```

src/ingestion/chunkers.py - dispatcher

1  CHUNK_SIZE_TOKENS = 400 # aprox. tokens por chunk (1 token = 4 chars)
2  CHUNK_OVERLAP_TOKENS = 80 # overlap entre chunks de textbooks

```

Captura del código real de `run_ingestion.py`: ejecuta los tres scripts de ingesta en orden y reporta el cierre del proceso.

```

19     from src.ingestion.ingest_multiclinsum import main as run_multiclinsum
20     run_multiclinsum()
21
22     if run_medmcqa:
23         from src.ingestion.ingest_medmcqa import main as run_mmc
24         run_mmc()
25
26     if run_medqa:
27         from src.ingestion.ingest_medqa import main as run_mq
28         run_mq()
29
30     print("-" * 60)
31     print("INGESTIÓN COMPLETA")
32     print("-" * 60)

```

```

src/ingestion/chunkers.py - dispatcher

1  CHUNK_SIZE_TOKENS = 400 # aprox. tokens por chunk (1 token = 4 chars)
2  CHUNK_OVERLAP_TOKENS = 80 # overlap entre chunks de textbooks
3
4  NO_CHUNK_SOURCES = {
5      "medmcqa", "medqa_us", "medqa_taiwan",
6      "medqa_mainland", "multiclinsum_summary",
7  }
8
9  def chunk_document(doc: Document) -> List[Document]:
10     """Decide qué estrategia aplicar según source_dataset."""
11     source = doc.metadata.get("source_dataset", "")
12
13     if source in NO_CHUNK_SOURCES:
14         return chunk_passthrough(doc)
15
16     if source == "multiclinsum_fulltext":
17         return chunk_paragraph_grouping(doc) # ~350 tok, overlap 1 párrafo
18
19     if source in ("textbook", "upload"):
20         return chunk_recursive_split(doc) # 400 tok / 80 overlap
21
22     return chunk_passthrough(doc) # fallback: source desconocido

```

`chunk_document()` decide entre *passthrough* (MedMCQA/MedQA, sin chunking), agrupación por párrafos (~350 tok, Multiclinsum fulltext) o `RecursiveCharacterTextSplitter` (400 tok / 80 overlap, textbooks/uploads), según `source_dataset`.

Observación técnica: el tamaño de chunk no es un parámetro arbitrario — la sección 8 de este informe muestra que re-chunkar `maternaqaes_lm` de ~879 a ~336 tokens promedio subió `faithfulness` un 170% (0,133–0,359), por lo que `CHUNK_SIZE_TOKENS=400` en `chunkers.py` refleja una decisión validada empíricamente, no un valor por defecto.

6.2 Indexación

Tampoco se ejecuta en vivo, por la misma razón que la ingestión. Se documenta con el código de `FAISSStore` y `embedder.py`.

```

src/ingestion/store.py - FAISSStore
1  """store.py - Constructor y lector del índice FAISS.
2
3  Archivos que gestiona en faiss_store/:
4  index.faiss      + vectores binarios
5  metadata.pkl     + dict { int_id + Document.metadata + text }
6  build_info.json  + auditoría: modelo, fecha, total de vectores
7  """
8
9  class FAISSStore:
10     """
11     Uso en ingestión:
12         store = FAISSStore.create_empty()
13         store.add_documents(chunks)
14         store.save()
15
16     Uso en retrieval:
17         store = FAISSStore.load()
18         results = store.search("¿qué es la preeclampsia?", k=5)
19     """
20
21     def is_active(meta: dict) -> bool:
22         """Un fragmento participa en el RAG salvo que esté
23         explícitamente desactivado.
24
25         El pipeline de ingestión NO escribe la clave 'active';
26         su ausencia significa ACTIVO. Nunca usar meta["active"]
27         sin default: descartaría el 100% del índice existente y
28         /chat seguiría respondiendo 200 con un fallback genérico,
29         sin ningún error visible.
30         """
31         return bool(meta.get("active", True))
32
33 src/ingestion/embedder.py - prefijos obligatorios (protocolo E5)
34
35 def embed_documents(texts: List[str], batch_size: int = 64, ...) -> np.ndarray:
36     """Aplica el prefijo 'passage: ' requerido por el modelo E5."""
37     model = _get_model()

```

`is_active()` — un fragmento participa en el RAG salvo que esté explícitamente desactivado; el docstring documenta por qué nunca se usa `meta["active"]` sin valor por defecto (descartaría el 100% del índice existente en silencio, ver también sección 9).

```

20
21 def is_active(meta: dict) -> bool:
22     """Un fragmento participa en el RAG salvo que esté
23     explícitamente desactivado.
24
25     El pipeline de ingestión NO escribe la clave 'active';
26     su ausencia significa ACTIVO. Nunca usar meta["active"]
27     sin default: descartaría el 100% del índice existente y
28     /chat seguiría respondiendo 200 con un fallback genérico,
29     sin ningún error visible.
30     """
31     return bool(meta.get("active", True))
32
33 src/ingestion/embedder.py - prefijos obligatorios (protocolo E5)
34
35 def embed_documents(texts: List[str], batch_size: int = 64, ...) -> np.ndarray:
36     """Aplica el prefijo 'passage: ' requerido por el modelo E5."""
37     model = _get_model()
38     prefixed = [f"passage: {t}" for t in texts]
39     ... # batches de 64, tqdm progress
40     matrix = np.vstack(all_vectors).astype(np.float32)
41     return _normalize(matrix) # l2 - coseno equivalente en IndexFlatIP
42
43 def embed_query(text: str) -> np.ndarray:
44     """Aplica el prefijo 'query: ' requerido por el modelo E5."""
45     model = _get_model()
46     prefixed = f"query: {text}"
47     vec = model.encode(prefixed, convert_to_numpy=True, normalize_embeddings=False)
48     return _normalize(vec.astype(np.float32).reshape(1, -1))[0]

```

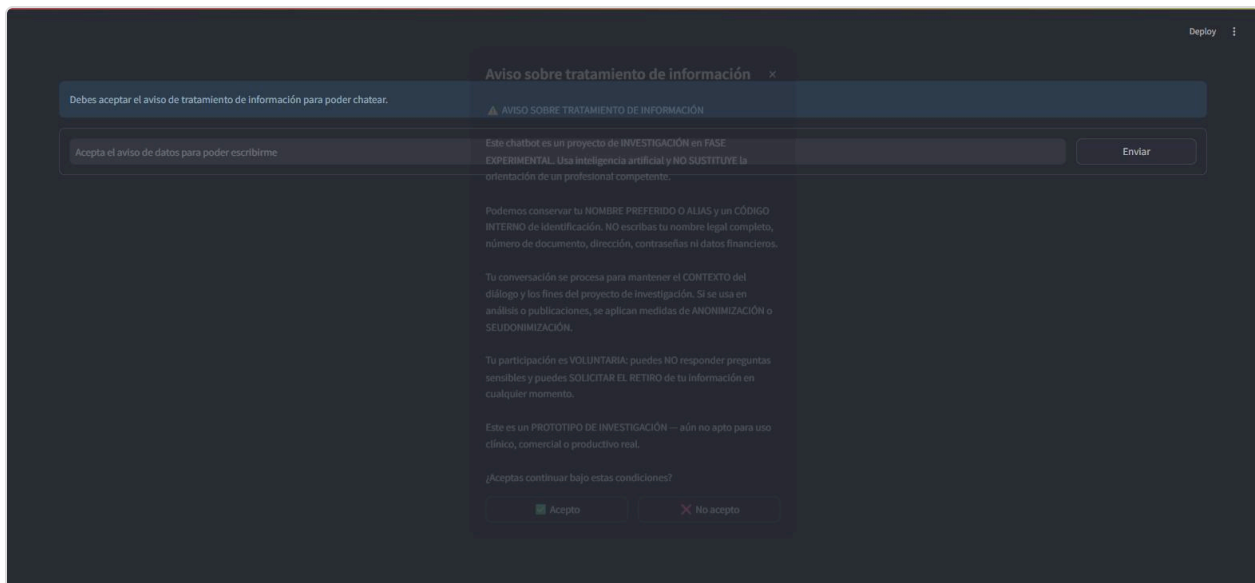
`embed_documents()` antepone `"passage: "` y `embed_query()` antepone `"query: "` — requisito del modelo `multilingual-e5-base`; sin estos prefijos los scores de similitud bajan ~15% (`qa_technical.md` Q5).

Observación técnica: `FAISSStore` usa `IndexFlatIP` (búsqueda exacta, sin aproximación) en vez de `IndexIVFFlat`. Con ~253k–380k vectores la búsqueda exacta resuelve en 20–50ms; la aproximación de IVFFlat solo se justifica a partir de millones de vectores (ver decisión en sección 9).

6.3 Retrieval

Ejecutado en vivo contra el índice de producción a través de la UI Streamlit, en varias sesiones independientes para cubrir: una gestante consultando directamente sobre distintos temas a lo largo de una conversación, el caso especial de un miembro de la red de apoyo consultando en representación de ella, y un flujo de clarificación completo.

Sesión nueva — aviso de tratamiento de datos:



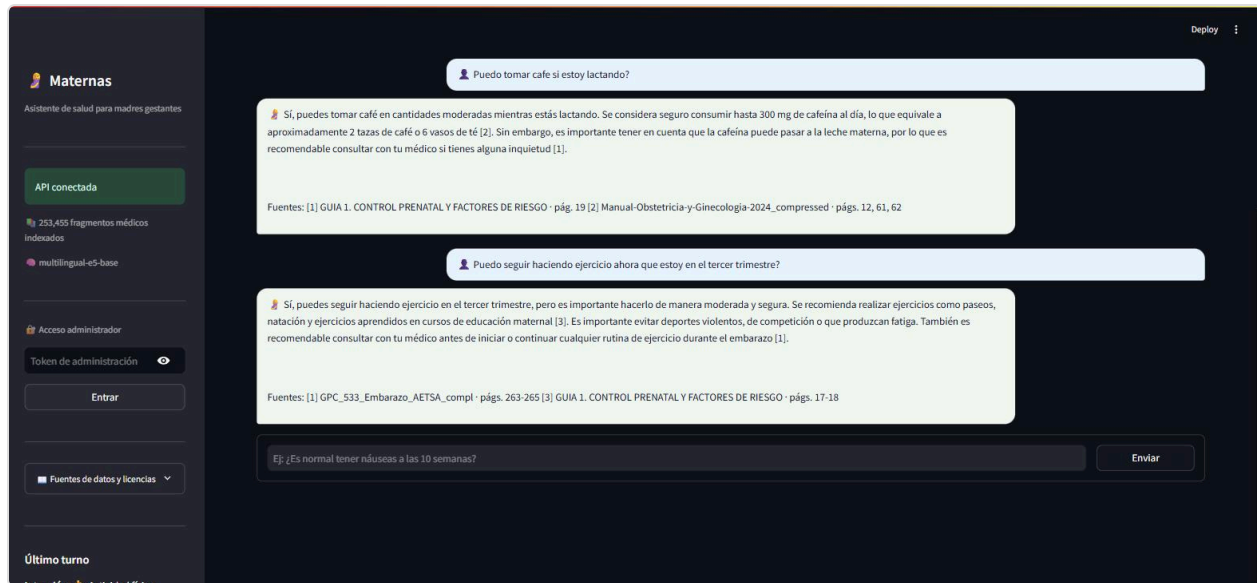
Cada sesión nueva de navegador (una pestaña nueva basta) abre este `st.dialog` antes de habilitar el chat. Sin aceptar, el campo de mensaje permanece deshabilitado.



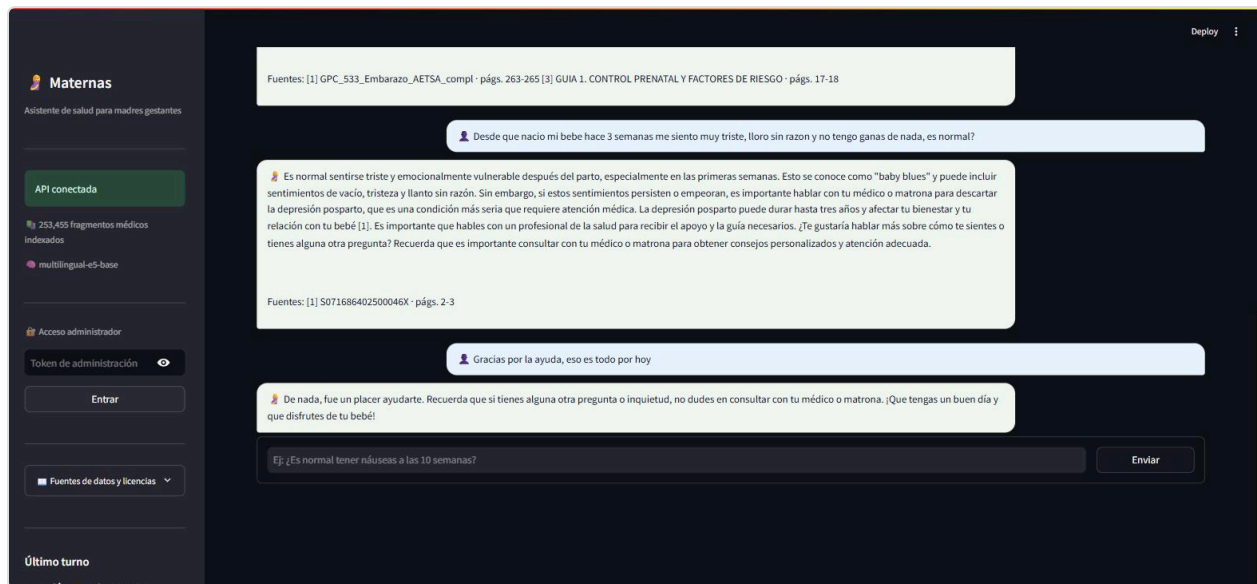
Estado inicial: barra lateral con estado de la API, conteo de fragmentos indexados (253.455) y modelo de embedding activo.

Caso 1 — Gestante, consulta directa sobre varios temas (misma conversación, continuada y concluida):

Se probó una única sesión con preguntas sobre nutrición, lactancia, actividad física y salud mental perinatal, dejando que la conversación avanzara varios turnos y cerrándola con una despedida — para observar cómo se comporta el sistema al cambiar de tema dentro del mismo hilo, en vez de solo con preguntas aisladas.



Tres turnos consecutivos de la misma sesión: alimentos a evitar (nutrición), cafeína en lactancia y ejercicio en tercer trimestre (actividad física) — cada uno recupera fuentes distintas y coherentes con el tema, con intención reclasificada turno a turno (ver barra lateral).



Turno sobre tristeza postparto: el sistema distingue "baby blues" de depresión posparto y recomienda seguimiento profesional (clasificado **riesgo=medium**, disparó una notificación real por correo — confirmado en los logs de la API, sin captura adicional para no sobrecargar el informe). La conversación se cierra con una despedida natural, reclasificada como intención **Administrativa**.

Caso 2 — Gestante, signos de alarma (riesgo alto):

The screenshot shows the 'Maternas — Asistente de Salud Materna' interface. The left sidebar displays the 'Último turno' (Last shift) with the following information:

- Intención:** Signos de alarma
- Riesgo:** ALTO
- Acción:** urgent_care
- Señales:**
 - cefalea
 - visión borrosa
- Fuentes recuperadas:**
 - GUÍA 1. CONTROL PRENATAL Y FACTORES DE RIESGO · págs. 17-18 · 0.854
 - Manual obstetrica_ginecologia · págs. 168-169 · 0.853
 - 63826549432ar4 · págs. 13-14 · 0.852
 - GAP: Control prenatal del embarazo normal · 6105 · págs. 15-16 · 0.850
 - 63826549432ar4 · págs. 10-11 · 0.850

The main content area shows a user input: 'Tengo 28 semanas de embarazo y me duele mucho la cabeza desde ayer, tambien veo puntos brillantes. Que debo hacer?'. The system response is: 'Debes buscar atención médica inmediata. El dolor de cabeza intenso y la visión de puntos brillantes pueden ser síntomas de preeclampsia, una condición que requiere evaluación y tratamiento urgentes. No esperes, acude a urgencias lo antes posible.' Below this, a follow-up question is shown: 'Ej: ¿Es normal tener náuseas a las 10 semanas?' with an 'Enviar' button.

Consulta real: "Tengo 28 semanas de embarazo y me duele mucho la cabeza desde ayer, tambien veo puntos brillantes." El panel lateral muestra en vivo: intención `Signos de alarma`, riesgo `ALTO`, acción `urgent_care`, señales detectadas (`cefalea`, `visión_borrosa`) y los 5 fragmentos recuperados con su score de similitud coseno (0,850–0,854).

Caso 3 — Red de apoyo (el esposo consulta en representación, riesgo medio→alto):

The screenshot shows the 'Maternas — Asistente de Salud Materna' interface. The left sidebar displays the 'Último turno' (Last shift) with the following information:

- Intención:** Signos de alarma
- Riesgo:** ALTO
- Acción:** urgent_care
- Señales:**
 - cefalea
 - visión borrosa
 - hipertensión
- Fuentes recuperadas:**
 - Guía completa Embarazo Parto 2013 · págs. 308-322 · 0.865
 - GINECOLOGÍA Y OBSTETRICIA · págs. 55-57 · 0.864
 - GUÍA 1. CONTROL PRENATAL Y FACTORES DE RIESGO · págs. 13-12 · 0.864
 - Capítulo 9 · págs. 4-6 · 0.863
 - Guía completa Embarazo Parto 2013 · ndex: 306-322 · 0.861

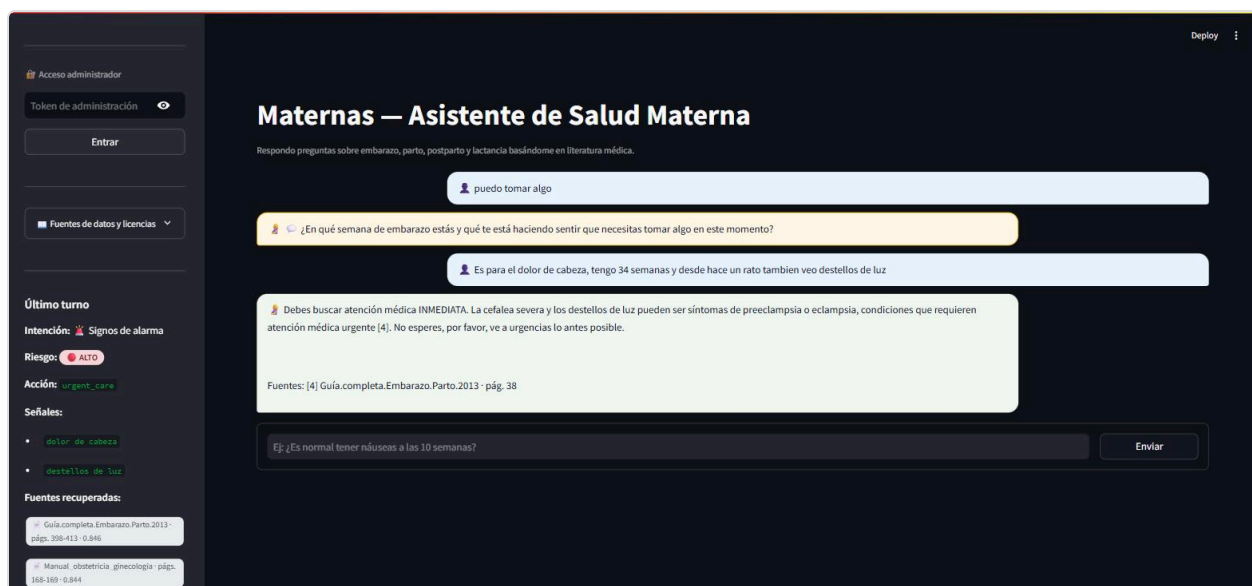
The main content area shows a user input: 'Hola, soy el esposo. Mi pareja tiene 20 semanas de embarazo y hoy le tomamos la presion, dio 140/90. Es normal o debemos preocuparnos?'. The system response is: 'La presión arterial de 140/90 en una mujer embarazada de 20 semanas puede ser un indicador de hipertensión. Según [1], se considera hipertensión en el embarazo una presión arterial mayor a 140/90 mmHg. Es importante que tu pareja consulte con su médico o matrona para evaluar la situación y determinar el curso de acción adecuado. No es necesario alarmarse, pero si es importante monitorear la presión arterial y realizar un seguimiento médico regular.' Below this, a follow-up question is shown: 'Ej: ¿Es normal tener náuseas a las 10 semanas?' with an 'Enviar' button.

Consulta real: "Hola, soy el esposo. Mi pareja tiene 20 semanas de embarazo y hoy le tomamos la presion, dio 140/90." El sistema recupera fuentes distintas y responde con el bloque `Fuentes: [1]` `Guía completa Embarazo Parto 2013 · págs. 308, 321, 322` — nombre de documento legible, no "fragmento [n]" genérico.

Caso 4 — Flujo de clarificación (con notificación disparada después de la clarificación):



Consulta real, deliberadamente vaga: "puedo tomar algo" (< 6 tokens, sin keyword de medicamento). El sistema no genera una respuesta médica a ciegas: pide contexto ("¿En qué semana de embarazo estás y qué te está haciendo sentir que necesitas tomar algo?").



Respuesta del usuario a la clarificación: "Es para el dolor de cabeza, tengo 34 semanas y desde hace un rato tambien veo destellos de luz." El sistema combina ambos turnos, clasifica **riesgo=ALTO** (señales: **dolor de cabeza**, **destellos de luz**) y responde citando la fuente — la pregunta original vaga por sí sola no hubiera bastado para esta clasificación.

Observación técnica: el retriever pide $k \times 10$ candidatos internos y filtra a **k=5** finales sobre **medmcqa** + **medqa_*** + **maternaqaes_lm** (Config D, sin BM25). Los scores observados en vivo (0,85–0,87) son consistentes con los reportados en **qa_technical.md** para el mismo tipo de consulta.

6.4 Clasificación de intención

Evidencia visible en las capturas de la sección 6.3 (`intent_classifier.py`) clasifica en el mismo turno que el retrieval). A lo largo de la conversación multi-tema del Caso 1 la intención se reclasificó correctamente en cada turno (`nutrición` → `lactancia` → `actividad_física` → `salud_mental_perinatal` → `administrativa`), y en el Caso 4 la intención se mantuvo en `signos_de_alarma` una vez el usuario aportó los síntomas concretos en la clarificación.

Observación técnica: `intent_classifier.py` expone 12 categorías fijas con tres niveles de fallback (zero-shot LLM → heurística por keywords → categoría por defecto), garantizando que el sistema siempre devuelva un intent válido incluso sin conexión a Groq.

6.5 Clasificación de riesgo

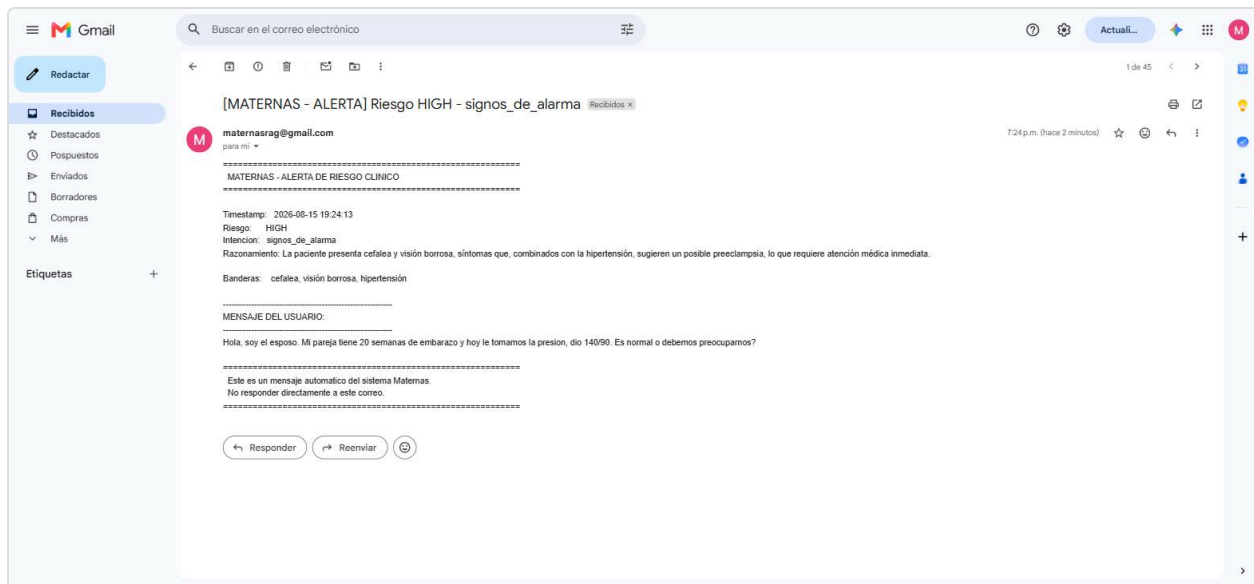
Misma evidencia visual (sección 6.3). El hallazgo relevante de la prueba en vivo se documenta en la sección 9.1: `detect_risk()` combina explícitamente los síntomas mencionados en turnos previos de la conversación con el mensaje actual antes de aplicar la heurística, para no evaluar cada síntoma aislado del cuadro clínico completo. Esto se observó dos veces en las pruebas: en el Caso 3 (la presión arterial se evaluó junto a la cefalea del turno anterior) y en el Caso 4 (la clarificación combinó "puedo tomar algo" con la cefalea y destellos de luz de la respuesta).

6.6 Redacción

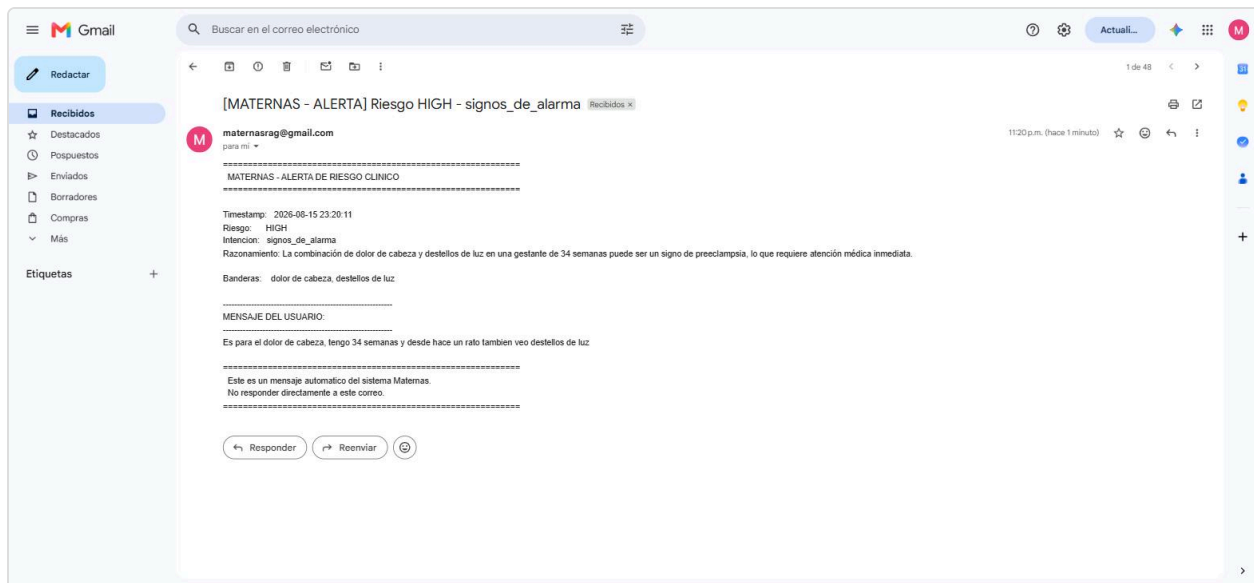
La generación de respuesta y el armado de citas (`citations.py`) se observan en todas las capturas anteriores: el LLM inserta marcadores `[n]` dentro del texto y `build_reference_block()` arma el bloque final `Fuentes:` agrupando por documento con nombre legible y páginas, no "fragmento [n]" genérico (ver Caso 3, sección 6.3).

6.7 Notificador de riesgo (SMTP)

Cada vez que `risk_detector.py` devuelve nivel `medium` o `high`, `NotifierSkill` dispara un correo de alerta real. Se verificó end-to-end contra la bandeja de entrada real (`maternasrag@gmail.com`), incluyendo el caso específico de una notificación disparada **después** de un intercambio de clarificación (Caso 4, sección 6.3).



Correo real recibido en `maternasrag@gmail.com` para el Caso 3 de la sección 6.3. El campo "Razonamiento" generado por el LLM combina las señales de ambos turnos de la conversación.



Correo real correspondiente al Caso 4: el "Mensaje del usuario" que se cita es la respuesta a la clarificación, y el razonamiento conecta explícitamente "dolor de cabeza" y "destellos de luz" con la preeclampsia — confirmando que la notificación solo se disparó una vez el sistema tuvo el cuadro completo, no con la pregunta vaga inicial.

Observación técnica: en la sesión de prueba de este informe se dispararon **4 notificaciones reales** (Caso 1 cierre, Caso 3, Caso 4, más una del Caso 2 en una prueba previa), confirmadas en los logs de la API (`[Chain] risk=... action=... → [Notifier] Email enviado a maternasrag@gmail.com`).

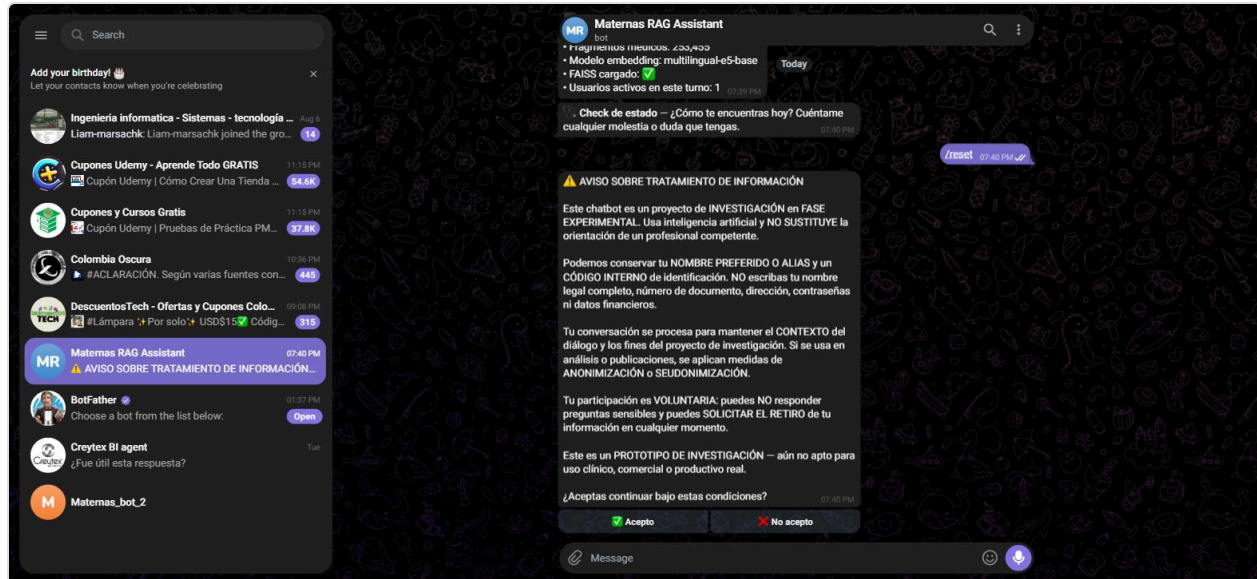
6.8 Streamlit

Ver capturas de consentimiento y chat inicial en la sección 6.3. La respuesta se transmite token a token vía `POST /chat/stream` (NDJSON: `status` → `meta` → `delta` → `done`), visible en vivo como los mensajes de progreso

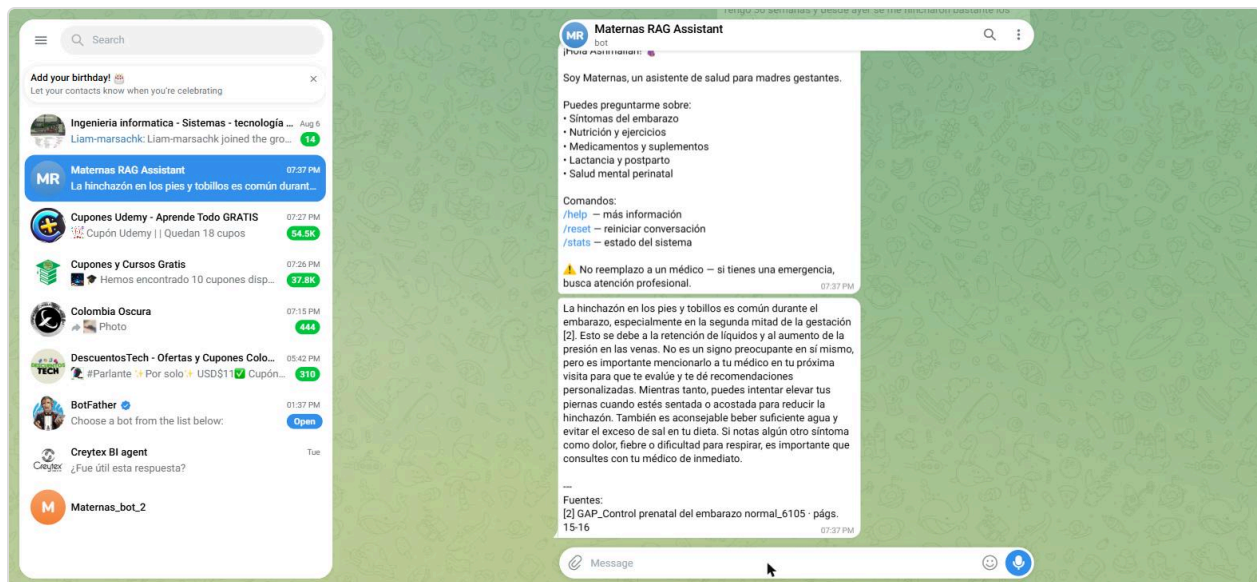
"Analizando tu mensaje..." → "Buscando en la base de conocimiento médico..." → "Redactando la respuesta..." antes de que el texto final aparezca, según `STAGE_LABELS` en `chat_view.py`.

6.9 Telegram

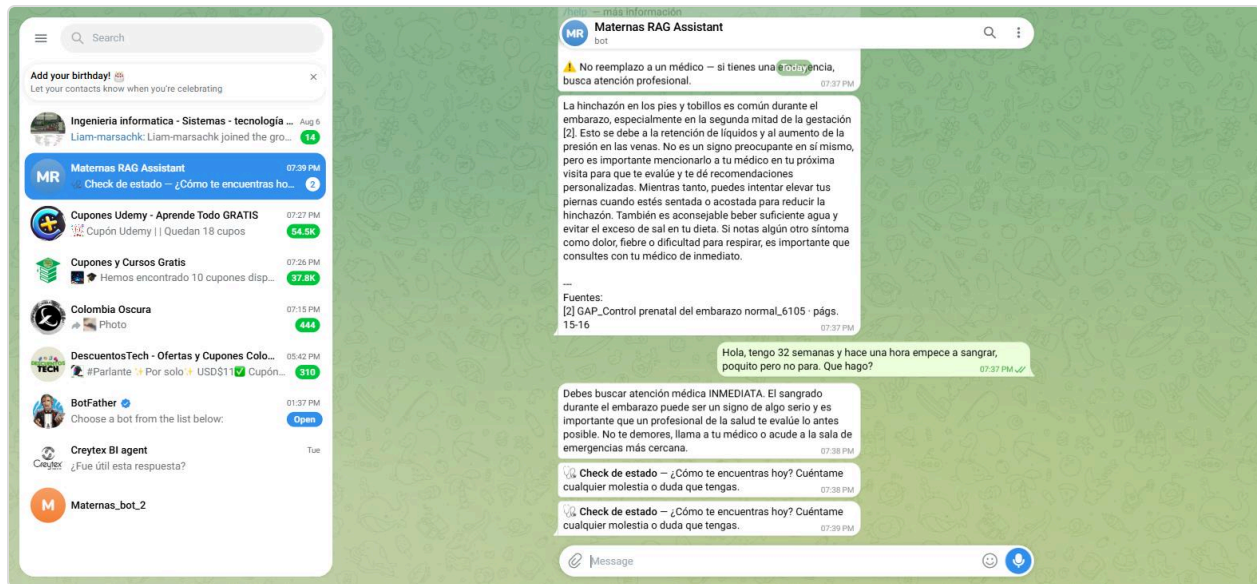
Bot real (`@MaternasAssistant_bot`) probado en vivo vía Telegram Web, con la API de producción corriendo en `localhost:8080`.



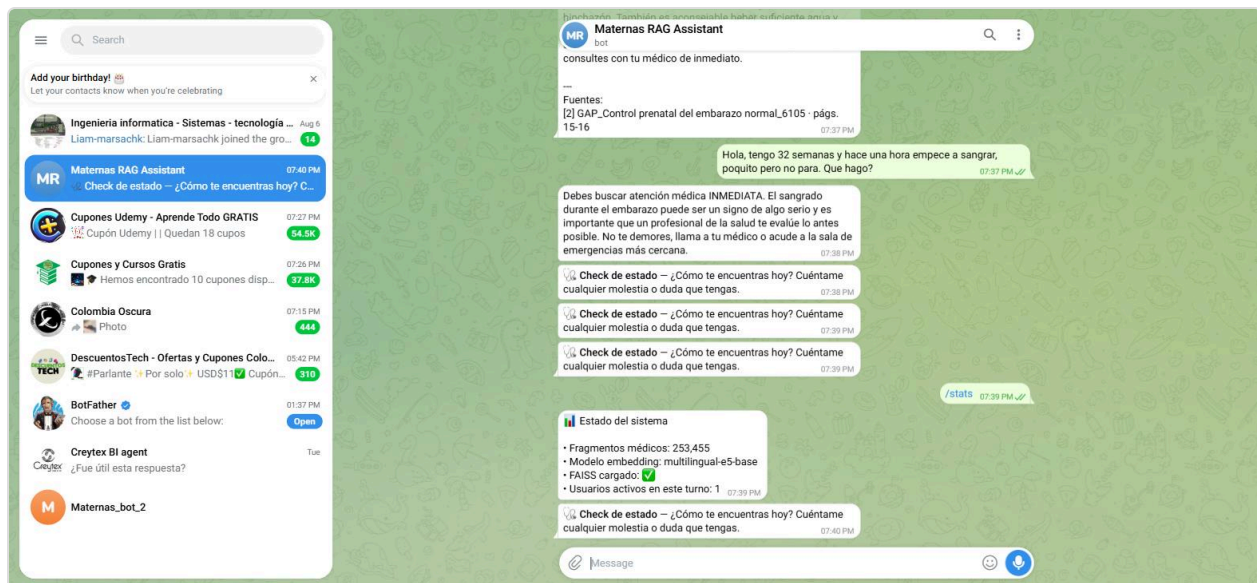
Con el proceso del bot recién reiniciado, el aviso de tratamiento de datos se muestra de nuevo antes de procesar cualquier mensaje — mismo comportamiento que Streamlit, pero como botones inline (`👍 Acepto` / `👎 No acepto`) en vez de un `st.dialog`.



Consulta real: "Tengo 30 semanas y desde ayer se me hincharon bastante los pies y las manos, es normal?" — clasificada `risk=Low`, respuesta educativa en texto plano con bloque `Fuentes:`.



Consulta real: "tengo 32 semanas y hace una hora empecé a sangrar, poquito pero no para" — clasificada **risk=high**. Debajo de la respuesta se ven los mensajes automáticos "Check de estado" del **JobQueue**, cuyo intervalo se redujo a los 30s configurados para riesgo alto (**STATUS_CHECK_INTERVAL_HIGH_SECONDS**).

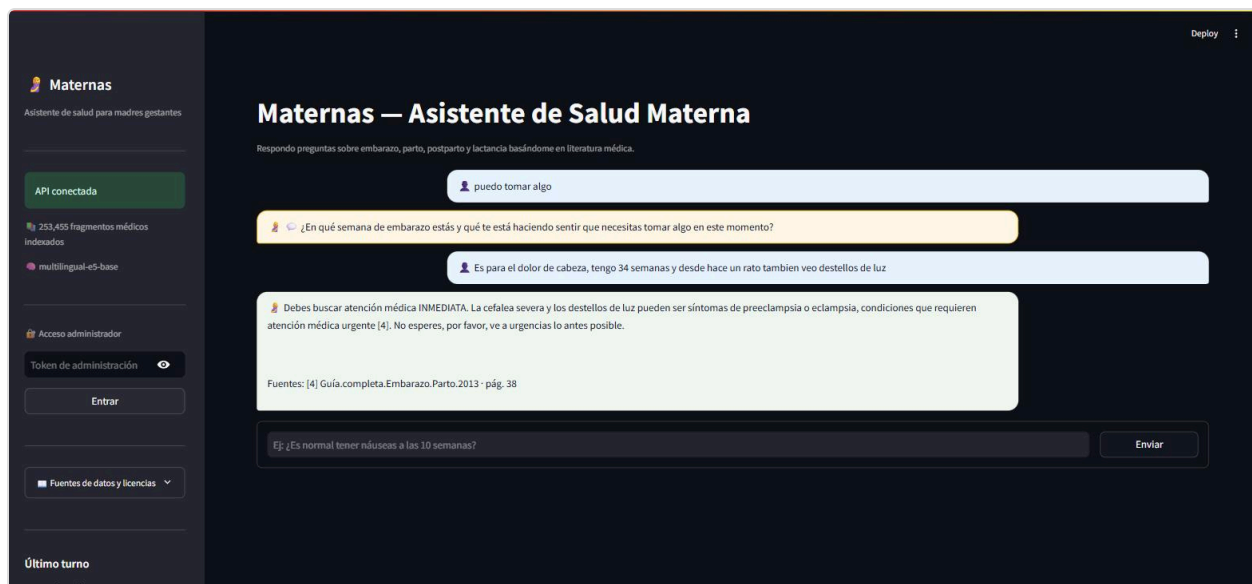


/stats consulta **GET /health** en vivo: 253.455 fragmentos médicos, modelo **multilingual-e5-base**, FAISS cargado **✓**, 1 usuario activo en el turno.

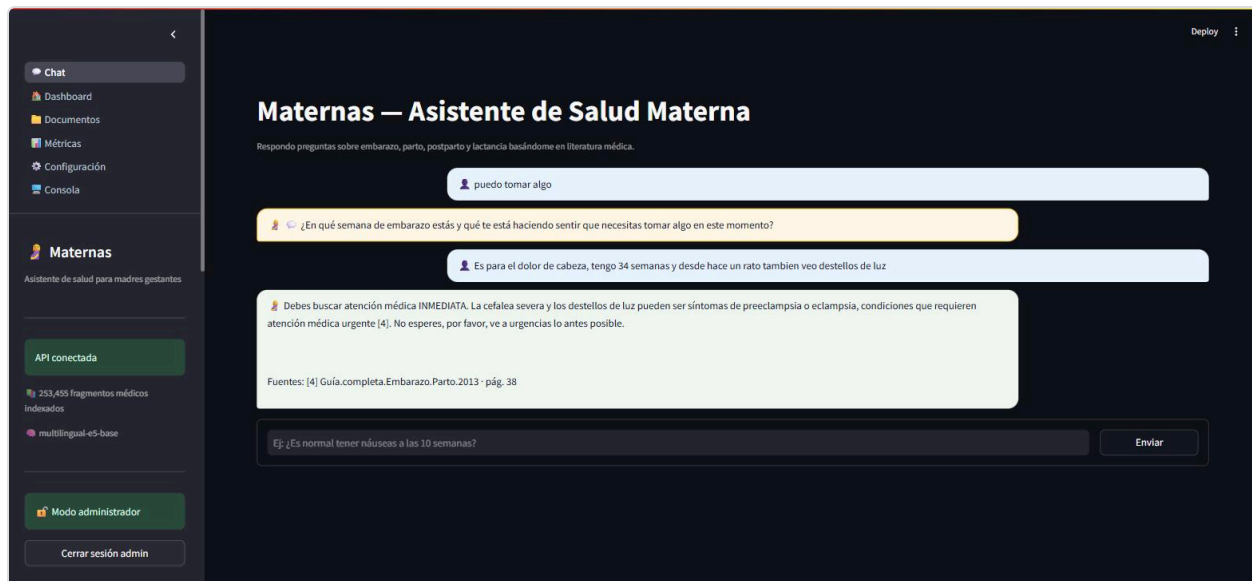
7. Panel de administración

Probado en vivo, con **ADMIN_API_TOKEN** real. El panel vive dentro de la misma app de Streamlit y solo aparece si la sesión se autenticó como admin.

7.1 Acceso

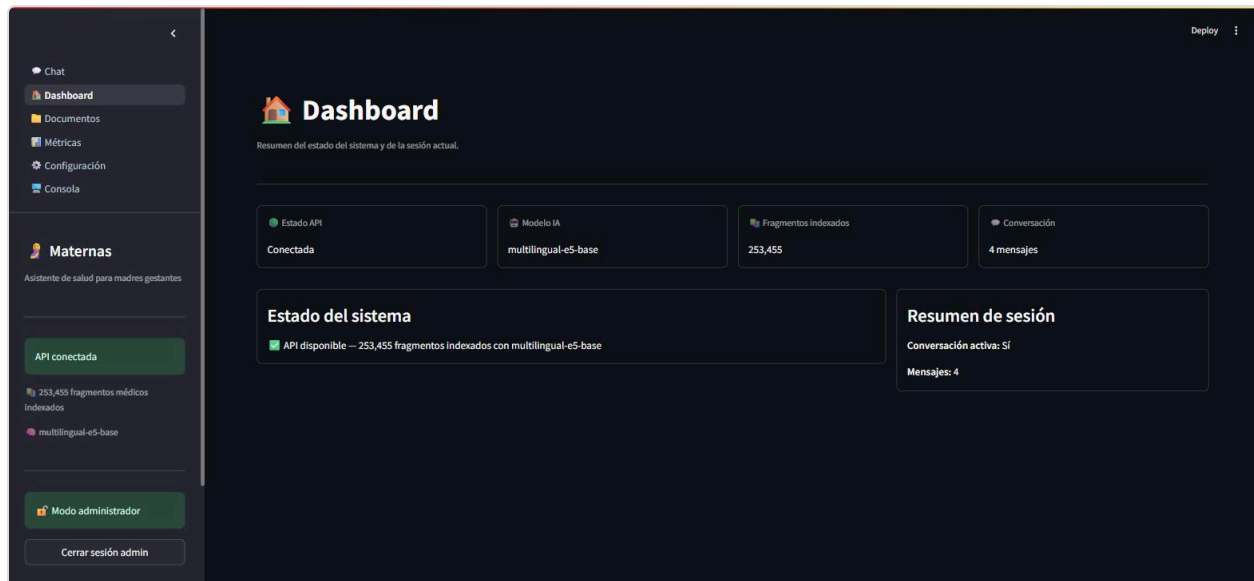


Campo "Token de administración" en la barra lateral, visible para cualquier sesión — sin token correcto, el panel permanece inalcanzable.



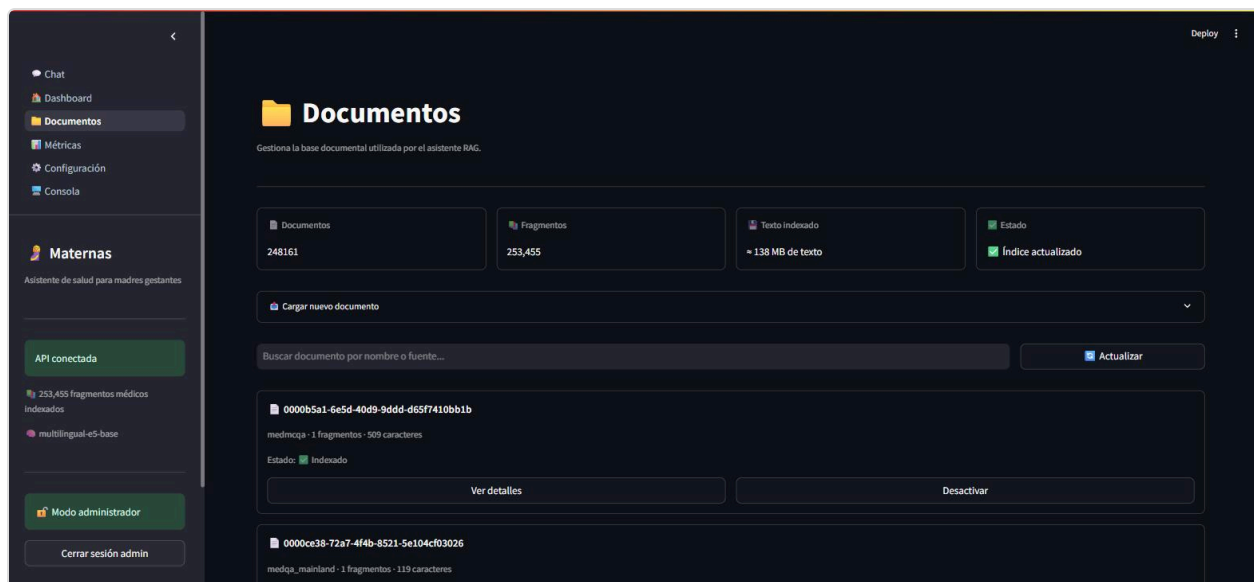
Tras ingresar el token real y pulsar "Entrar", la navegación lateral revela cinco páginas nuevas: Dashboard, Documentos, Métricas, Configuración y Consola.

7.2 Dashboard



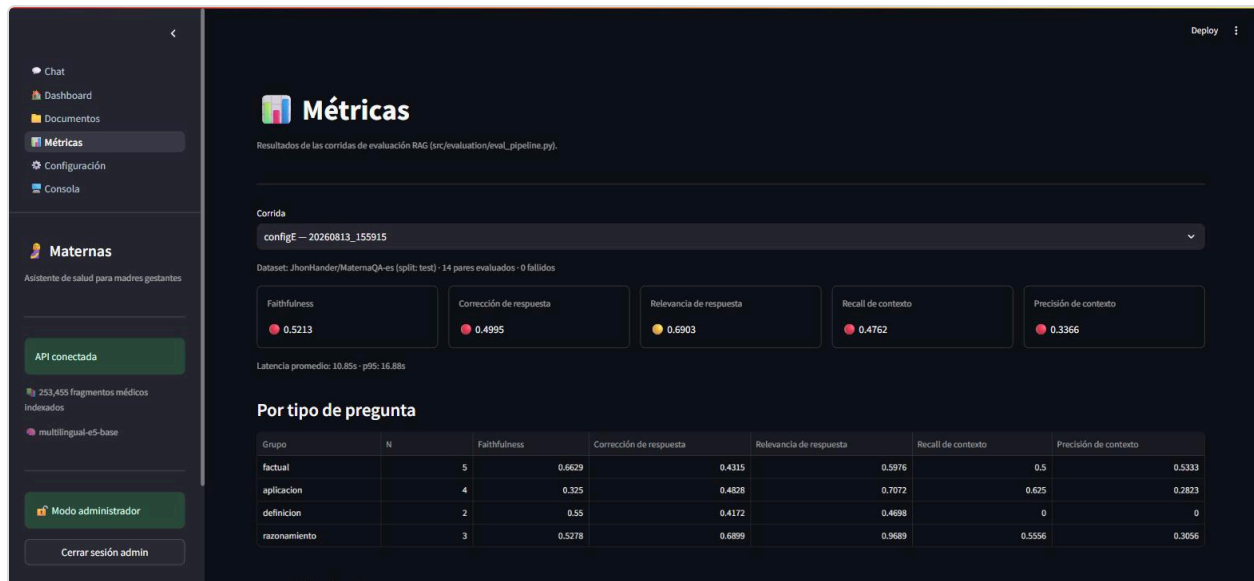
Resumen de sesión y estado del sistema: API conectada, 253.455 fragmentos, modelo activo y contador de mensajes de la sesión actual.

7.3 Documentos



Vista agregada real del índice: 248.161 documentos, 253.455 fragmentos, ~138 MB de texto indexado. Permite buscar, ver detalle paginado, activar/desactivar y subir documentos nuevos.

7.4 Métricas



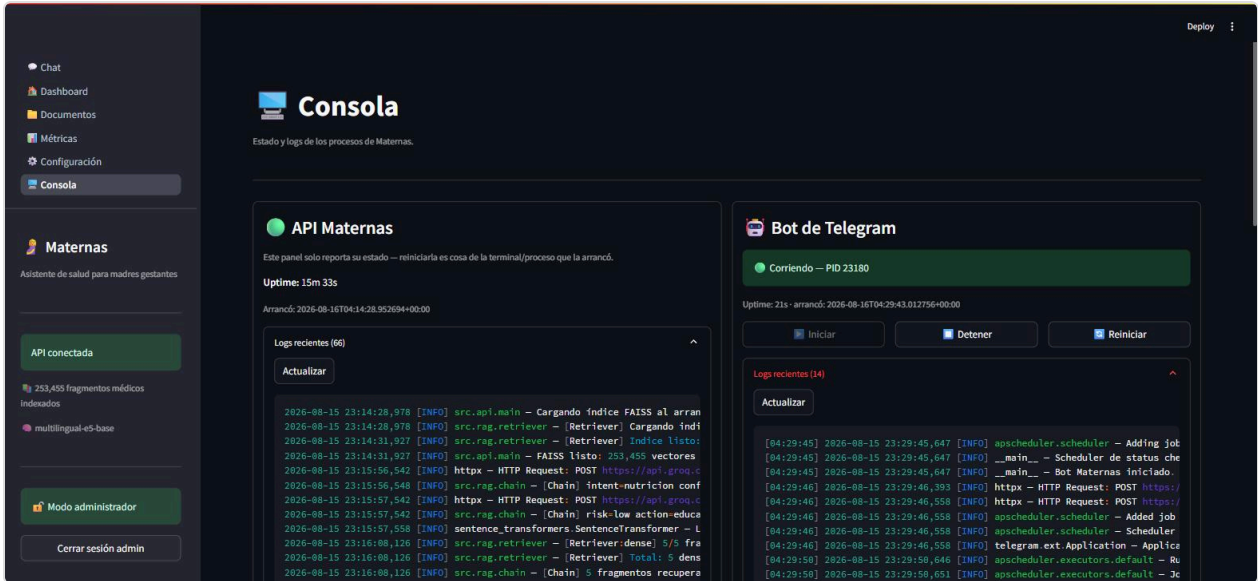
Lectura de una corrida ya generada (`configE - 20260813_155915`) sin recomputar nada — coincide con `evaluation_reports/eval_report_configE_20260813_155915.md`.

7.5 Configuración



Confirma en vivo la configuración activa: embedding `multilingual-e5-base` en `cuda`, LLM `llama-3.3-70b-versatile`, y las fuentes activas del índice denso (`maternaqaes_lm`, `medmcqa`, `medqa_mainland`, `medqa_taiwan`, `medqa_us`, `upload`) — Config D, sin `textbook` ni `multiclinsum`.

7.6 Consola — API y bot de Telegram



Tras pulsar "Iniciar" en el panel del bot, la API lo lanza como subprocesso hijo (`PID 23180` en la captura) y expone sus logs en tiempo real junto a los de la propia API — ambos con líneas reales tomadas de la sesión de pruebas de este informe.

Observación técnica: el estado del bot que reporta este panel es el que administra `bot_supervisor.py` (`subprocess.Popen`), no cualquier proceso de `maternas_bot.py` corriendo en la máquina — un bot iniciado manualmente por fuera de la API (como se hizo en las secciones 6.3–6.9 para poder probarlo con el `.env` de este informe) aparece como "Detenido" en este panel hasta que se administra a través de él, consistente con el diseño de estado por-proceso documentado en la sección 9.3.

8. Resultados y evaluación

Framework: **Ragas**, benchmark **MaternaQA-es** (split test, 328 pares QA en español), evaluación en dos fases con modelos independientes (Groq `llama-3.3-70b-versatile` genera, Cerebras `gemma-4-31b` evalúa) para evitar que el mismo modelo se autoevalúe.

Métricas de la configuración de producción (Config D)

Métrica	Config D (producción)	Baseline MaternaQA-es (test)	Estado
<code>faithfulness</code>	0,4973	0,7132	●
<code>answer_correctness</code>	0,5507	N/A	●
<code>answer_relevancy</code>	0,7328	0,5583	● (supera baseline)
<code>context_recall</code>	0,4524	N/A	●

Métrica	Config D (producción)	Baseline MaternaQA-es (test)	Estado
<code>context_precision</code>	0,3599	N/A	●
<code>latency_avg_s</code>	9,58 s	—	—
<code>latency_p95_s</code>	15,93 s	—	—

N = 14 pares evaluados, 0 fallidos en generación. Fuente:

`evaluation_reports/eval_report_configD_20260813_151914.md`.

Progresión histórica por configuración

Config	N	Faithfulness	Ans. Correct.	Ans. Relev.	Ctx. Recall	Ctx. Prec.	Lat. (s)
A — FAISS puro	15	0,162	0,350	0,635	0,000	0,000	11,35
B — FAISS+BM25	15	0,228	0,338	0,631	0,000	0,000	10,36
C v3 — +corpus ES	14	0,456	0,532	0,816	0,452	0,388	10,23
D — producción actual	14	0,497	0,551	0,733	0,452	0,360	9,58
E — D + HyDE (descartada)	14	0,521	0,500	0,690	0,476	0,337	10,85
Baseline MaternaQA-es (test)	—	0,713	—	0,558	—	—	—

Interpretación

- `answer_relevancy` (0,733) **supera el baseline publicado** (0,558) en la configuración de producción — el modelo generador mantiene relevancia temática de forma consistente en todas las configuraciones probadas.
- `context_recall` y `context_precision` permanecen por debajo del ideal porque el índice de producción no contiene los 3 PDFs exactos que originaron el benchmark de evaluación (excluidos deliberadamente para evitar data leakage, ver `DOCUMENTACION.md` sección 4).
- El tamaño de muestra (14 pares) implica una desviación estándar considerable (~0,25, según `eval_runbook.md`); los deltas entre C, D y E caen dentro de ese margen de ruido — por eso ni la remoción de `textbook` / `multiclinsum` ni HyDE se consideraron cambios "positivos" o "negativos" de forma concluyente, sino decisiones evaluadas por su relación costo/beneficio (licencia y latencia, respectivamente).
- **Pendiente / no medido a la fecha de este informe:** ampliación de la muestra de evaluación a 30 pares para reducir la varianza (backlog abierto, ver sección 10).

9. Problemas encontrados y decisiones técnicas

9.1 Riesgo clínico consciente del historial (verificado en prueba en vivo)

`detect_risk()` (`src/classifiers/risk_detector.py`, líneas 330–359) combina explícitamente los síntomas de turnos previos con el mensaje actual antes de aplicar la heurística:

"Cuando se provee `history`, los síntomas mencionados en turnos anteriores de la misma conversación (...) se combinan con el mensaje actual, para no evaluar cada síntoma aislado del resto del cuadro clínico."

Esto se confirmó en vivo repetidas veces en la sección 6.3: una pregunta sobre presión arterial de 140/90 que, aislada, podría clasificarse riesgo medio, se evaluó `riesgo=high` al combinarse con la cefalea mencionada antes en la misma sesión; y una clarificación deliberadamente vaga ("puedo tomar algo") terminó en `riesgo=ALTO` en cuanto la respuesta reveló cefalea y destellos de luz — comportamiento clínicamente conservador e intencional, no un efecto colateral de la UI (se verificó que `chat_view.py` solo lee `meta[-1]`, el último turno; la acumulación de contexto ocurre en el backend).

9.2 Falta de documentación médica obstetrica en el proyecto

Con solamente documentos obstetricos chunkerizados provenientes de `maternasqa-es` el desarrollo puede no tener el contexto suficiente para responder todas las preguntas o casos más complejos de maternidad.

9.3 Decisiones de arquitectura (trade-offs)

Decisión	Alternativas consideradas	Razón elegida
FAISS <code>IndexFlatIP</code>	<code>IndexIVFFlat</code> (aproximado)	~253k–380k vectores → búsqueda exacta en 20–50ms; IVFFlat solo se justifica en millones de vectores
<code>multilingual-e5-base</code> (768 dims)	MiniLM, BGE	Soporte nativo ES/EN/ZH en un solo modelo; prefijos obligatorios <code>query:</code> / <code>passage:</code>
Groq <code>llama-3.3-70b</code> para generación	GPT-4, LLM local	Costo cero (100k tok/día), latencia baja (2–4s), calidad en español suficiente
Cerebras <code>gemma-4-31b</code> como judge Ragas	Groq <code>llama-3.3-70b</code> , <code>llama-3.1-8b</code>	Sin límite diario de tokens; llama-3.3-70b agotaba cuota en 12 pares, llama-3.1-8b fallaba en el parser de Ragas
Chunking ~336 tok para <code>maternaqaes_lm</code>	~879 tok originales	<code>faithfulness</code> 0,133→0,359 (+170%) al re-chunkar — el juez Ragas localiza mejor afirmaciones en fragmentos atómicos

Decisión	Alternativas consideradas	Razón elegida
Heurística + LLM en cascada para riesgo	LLM para todo	Heurística: latencia ~0ms, determinismo y sin costo de API para casos obvios (hemorragia, convulsión)
Remoción de <code>textbook</code> / <code>multiclinsum</code> (Config D)	Mantenerlos indexados	Riesgo de licencia no resuelto; impacto en métricas medido primero y confirmado dentro del ruido estadístico
HyDE descartado (Config E)	Adoptarlo en producción	Sin mejora concluyente; +1,27s de latencia y agotamiento de cuota Groq a mitad de la evaluación
Bot de Telegram como subproceso hijo de la API	systemd/Docker/supervisor externo	Proyecto en una sola máquina sin orquestador; suficiente para iniciar/detener/reiniciar desde el panel admin

9.4 Limitaciones conocidas

Limitación	Impacto
CORS abierto a <code>*</code>	Cualquier origen puede consumir <code>/health</code> , <code>/chat</code> , <code>/classify</code> (los endpoints admin sí están protegidos)
Cuota de 100k tok/día en Groq	Limita evaluaciones largas; mitigado parcialmente con segunda API key
<code>faithfulness</code> 26pp por debajo del baseline	Atribuible a ausencia de los PDFs exactos del benchmark en el índice de producción
<code>--workers 1</code> obligatorio en uvicorn	<code>FAISSStore</code> , sus locks y <code>bot_supervisor</code> son estado por-proceso; múltiples workers divergirían
Sin despliegue automatizado	No hay Dockerfile ni CI/CD; ejecución manual documentada en la sección 5
Sin entrada de voz ni imágenes	El sistema solo acepta texto — ni la UI Streamlit ni el bot de Telegram procesan notas de voz, fotos ni archivos adjuntos
Verificación de nuevas fuentes provenientes del panel Admin	Cualquier persona con clave admin puede subir TXTs al índice del proyecto sin revisar su licencia

10. Conclusiones y próximos pasos

Maternas cumple, a la fecha de este informe, con el objetivo funcional definido en el segundo entregable del proyecto: un chatbot RAG que clasifica intención y riesgo, recupera contexto de literatura médica indexada, genera respuestas con citas y escala automáticamente los casos de riesgo medio/alto — verificado en este

informe con pruebas reales sobre las tres interfaces (API, Streamlit, Telegram), el panel de administración completo, y confirmación end-to-end del canal de notificación por correo, incluyendo el caso de una notificación disparada después de un flujo de clarificación. La arquitectura de retrieval fue objeto de cinco iteraciones medidas experimentalmente, y las decisiones más recientes (remoción de datasets con licencia no resuelta, descarte de HyDE, simplificación del badge de riesgo en Telegram) siguieron el mismo protocolo: medir o justificar antes de decidir.

Las brechas más relevantes frente al sistema de referencia (`faithfulness` , `context_recall`) tienen una causa raíz identificada y documentada (ausencia de los PDFs exactos del benchmark en el índice de producción, por diseño, para evitar data leakage), no un defecto no diagnosticado.

Próximos pasos (backlog abierto, `DOCUMENTACION.md` sección 17)

- ☐ Reranker cross-encoder local (`BAAI/bge-reranker-v2-m3`)
- ☐ System prompt más restrictivo ("no tengo información suficiente" explícito)
- ☐ Web search skill (fallback Tavily)
- ☐ Dockerización de API + Streamlit
- ☐ Corpus obstetricos ampliado (guías OMS, FIGO, guías nacionales adicionales)
- ☐ Ampliar muestra de evaluación a 30 pares para reducir varianza
- ☐ Soporte de entrada por voz o imagen (fuera de alcance actual)

11. Anexos

11.1 Endpoints de la API

Método	Ruta	Descripción	Auth
<code>GET</code>	<code>/health</code>	Estado del servicio, vectores cargados, modelo activo	No
<code>POST</code>	<code>/chat</code>	Turno completo RAG	No
<code>POST</code>	<code>/chat/stream</code>	Igual que <code>/chat</code> , NDJSON token a token	No
<code>POST</code>	<code>/classify</code>	Solo clasificadores (intent + risk)	No
<code>GET/PATCH</code>	<code>/documents*</code>	Gestión del índice FAISS en caliente	<code>X-Admin-Token</code>
<code>GET/PATCH</code>	<code>/admin/config</code>	Configuración editable de 10 variables <code>.env</code>	<code>X-Admin-Token</code>
<code>GET</code>	<code>/admin/evaluations*</code>	Lectura de corridas Ragas ya generadas	<code>X-Admin-Token</code>
<code>GET/POST</code>	<code>/admin/bot/*</code>	Control del subprocesso del bot de Telegram	<code>X-Admin-Token</code>

11.2 Variables de entorno relevantes

Variable	Rol
<code>GROQ_API_KEY</code> / <code>GROQ_API_KEY_2</code>	LLM principal / segunda clave para evaluación Ragas
<code>CEREBRAS_KEY</code>	Judge de evaluación Ragas
<code>EMBEDDING_MODEL</code> , <code>EMBEDDING_DEVICE</code>	Modelo y dispositivo de embedding
<code>RAG_TOP_K</code>	Fragmentos recuperados por consulta (default 5)
<code>ADMIN_API_TOKEN</code>	Protege el panel administrativo (fail-closed si vacío)
<code>TELEGRAM_BOT_TOKEN</code>	Token del bot (BotFather)
<code>NOTIFIER_*</code>	Configuración SMTP del canal de alertas
<code>STATUS_CHECK_INTERVAL_LOW/MEDIUM/HIGH_SECONDS</code>	Cadencia del scheduler de check-ins según riesgo

11.3 Referencias a documentación completa

- `docs/DOCUMENTACION.md` — documentación técnica completa del sistema (18 secciones)
- `foragents/qa_technical.md` — 32 preguntas técnicas resueltas, incluyendo las decisiones de licencia (Q28, Q31) y HyDE (Q32) citadas en este informe
- `foragents/retrieval_arquitecturas_configs.md` — detalle de las configuraciones de retrieval A y B
- `evaluation_reports/` — reportes crudos y agregados de todas las corridas Ragas

Informe generado a partir del código, documentación y evidencia en vivo del repositorio `maternas-rag` (commit `5707d01`). Todo dato reportado es trazable al código fuente, la documentación existente, los reportes de `evaluation_reports/`, el historial de git o la evidencia en vivo listada arriba — sin datos inventados.